



DEPARTMENT OF
COMPUTER SCIENCE

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

MASTER IN INTEGRATED MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon

Draft: February 2, 2026

CHALLENGES IN LEARNING AND TEACHING SOFTWARE MODELLING: A SOCIO-TECHNICAL GROUNDED THEORY APPROACH

YAROSLAV HAYDUK

Bachelor in Computer Science and Engineering

Supervisor: Miguel Goulão

Professor, NOVA University Lisbon

Co-supervisor: Grischa Liebel

Associate Professor, Reykjavik University

ABSTRACT

Software and conceptual modelling are widely taught in software engineering and related computing programmes, yet students often experience modelling as either an essential aid for understanding and communication or as burdensome “extra documentation”. Prior research on modelling education offers valuable insights but remains fragmented across modelling families and communities. Bloom-based frameworks and modelling-specific taxonomies help articulate learning outcomes and competencies, while empirical studies of specific modelling methods highlight recurring difficulties, such as understanding construct semantics, applying modelling to requirements, and judging model quality. However, there remains a need to consolidate these perspectives into an integrated, empirically grounded understanding of modelling education challenges and support needs that reflects both student and instructor viewpoints across educational contexts.

This thesis investigates how software modelling is experienced, taught, and learned in higher education, treating modelling as a socio-technical activity shaped by conceptual understanding, procedural strategies, tool mediation, and course design constraints. The study addresses three research questions: (RQ1) what challenges students and instructors perceive in learning and teaching modelling and how these challenges manifest across modelling activities (e.g., interpreting, creating, refining, evaluating); (RQ2) which strategies are used to address these challenges and what pedagogical and tool-related support needs are identified; and (RQ3) how socio-technical factors such as tools, domains, feedback practices, and course structures shape modelling experiences.

Methodologically, the thesis adopts a qualitative, theory-building approach based on Grounded Theory, informed by Socio-Technical Grounded Theory to explicitly attend to interactions between social and technical elements. Data are collected through semi-structured interviews complemented by activity-centered elicitation grounded in concrete modelling tasks or artefacts. Bloom’s revised taxonomy is used as a sensitising concept to support interpretation and communication of the cognitive nature of challenges without constraining the grounded analysis. The expected outcome is an empirically grounded account of modelling education challenges and support needs that can inform the design of teaching practices and educational tools aimed at fostering modelling competence in

realistic learning contexts.

Keywords: software modelling education · conceptual modelling · grounded theory · socio-technical perspective · modelling competence

RESUMO

A modelação de software e a modelação conceptual são amplamente ensinadas em cursos de Engenharia de Software e áreas relacionadas, mas os estudantes frequentemente percebem a modelação como um apoio essencial à compreensão e comunicação ou, pelo contrário, como “documentação extra” que compete com o tempo de implementação. A investigação existente em educação para a modelação oferece contributos relevantes, mas permanece fragmentada entre diferentes famílias de modelação e comunidades científicas. Por um lado, enquadramentos baseados na taxonomia revista de Bloom e taxonomias específicas de modelação ajudam a explicitar resultados de aprendizagem e competências; por outro, estudos empíricos centrados em métodos particulares evidenciam dificuldades recorrentes, como compreender a semântica dos construtos, aplicar a modelação a requisitos e julgar a qualidade dos modelos. Ainda assim, permanece a necessidade de consolidar estas perspetivas numa compreensão integrada e empiricamente fundamentada dos desafios e necessidades de apoio na educação para a modelação, incorporando simultaneamente os pontos de vista de estudantes e docentes em diferentes contextos educativos.

Esta tese investiga como a modelação de software é experienciada, ensinada e aprendida no ensino superior, tratando a modelação como uma atividade socio-técnica moldada pela compreensão conceptual, por estratégias procedimentais, pela mediação de ferramentas e por constrangimentos de desenho curricular. O estudo é guiado por três questões de investigação: (RQ1) que desafios estudantes e docentes percebem na aprendizagem e no ensino da modelação, e como esses desafios se manifestam ao longo de diferentes atividades (por exemplo, interpretar, criar, refinar e avaliar modelos); (RQ2) que estratégias são usadas para lidar com esses desafios e que necessidades de apoio (pedagógicas e relacionadas com ferramentas) são identificadas; e (RQ3) de que forma fatores socio-técnicos, como ferramentas, desenho de unidades curriculares, domínios de trabalho e práticas de feedback, influenciam as experiências de aprendizagem e ensino da modelação.

Do ponto de vista metodológico, a tese adota uma abordagem qualitativa e de construção de teoria baseada em Grounded Theory, informada pela Socio-Technical Grounded Theory para atender explicitamente às interações entre elementos sociais e técnicos. Os

dados são recolhidos através de entrevistas semi-estruturadas, complementadas por elicitação centrada em atividades ancorada em tarefas ou artefactos concretos de modelação. A taxonomia revista de Bloom é usada como conceito sensibilizador para apoiar a interpretação e comunicação da natureza cognitiva dos desafios, sem constranger a análise fundamentada. O resultado esperado é um entendimento empiricamente fundamentado dos desafios e necessidades de apoio na educação para a modelação, com implicações para o desenho de práticas pedagógicas e de ferramentas educativas que promovam a competência de modelação em contextos realistas.

Palavras-chave: educação em modelação de software · modelação conceptual · grounded theory · perspectiva socio-técnica · competência de modelação

CONTENTS

1	Introduction	1
1.1	Motivation	1
1.2	Problem statement and research gap	1
1.3	Aim and research questions	2
1.4	Research approach overview	2
1.5	Expected contributions	3
1.6	Thesis structure	3
2	Background	4
2.1	Software modelling & conceptual modelling	4
2.1.1	Purpose and role of modelling in software engineering	4
2.1.2	Conceptual modelling as a broader discipline	4
2.1.3	Modelling methods, languages, and artefacts	5
2.1.4	Implications for the scope of this thesis	5
2.2	Modelling education	6
2.2.1	Overview of conceptual modelling education	6
2.2.2	Bloom-based perspectives on modelling learning outcomes	6
2.2.3	Process and goal modelling education	7
2.2.4	Modelling, tools, and model-driven engineering in education	7
2.2.5	Competence-oriented and human-centred perspectives	8
2.2.6	Implications for this thesis	9
2.3	Grounded Theory	9
2.3.1	Overview and key characteristics	9
2.3.2	Grounded Theory in software engineering	10
2.3.3	Socio-Technical Grounded Theory	10
2.3.4	Implications for this thesis	11
2.4	Bloom’s Taxonomy and learning outcomes	11
2.4.1	Overview of Bloom’s revised taxonomy	11
2.4.2	Learning outcomes in computing and software engineering	12

2.4.3	Bloom-based formulations of modelling learning outcomes	12
2.4.4	Bloom, competencies, and this thesis	13
2.5	Human factors in modelling and SE education	13
2.5.1	Motivation, engagement, and the “value” of modelling	13
2.5.2	Inclusion, domain diversity, and participation barriers	14
2.5.3	Tools as mediators of learning experience	14
2.5.4	Authenticity, experiential learning, and social dynamics	15
2.5.5	Competence development and human-centred educational design	15
2.5.6	Implications for this thesis	15
3	Related Work	17
3.1	Conceptual modelling education landscape	17
3.2	Modelling frameworks	17
3.2.1	CaMeLOT: a framework for conceptual data modelling learning outcomes	18
3.2.2	Domain Modelling in Bloom: empirical analysis of assessment practices	18
3.2.3	Teaching conceptual modelling and metamodeling	18
3.3	Empirical studies on modelling learning challenges	18
3.3.1	iStar learning challenges and support requirements	19
3.3.2	Process modelling education and the need for empirical validation	19
3.4	Tools and model-driven engineering in education	19
3.4.1	Perception of tools and UML in MDE education	19
3.4.2	Human factors in model-driven engineering	20
3.5	Human factors and learning context in modelling education	20
3.5.1	Domain diversity, motivation, and inclusion	20
3.5.2	Experiential learning and authenticity	20
3.6	Competence-oriented agendas and methodological precedents	21
3.6.1	Competency identification and development in SE education	21
3.6.2	Grounded Theory in software engineering research	21
3.7	Summary	21
4	Methodology	22
4.1	Research design and rationale	22
4.2	Research questions	22
4.3	Study context and participants	23
4.4	Data collection	23
4.4.1	Instruments and pilot	23
4.4.2	Activity-centered elicitation: modelling task	24
4.4.3	Interviews	24
4.4.4	Surveys	24

4.5	Data analysis	24
4.5.1	Grounded Theory variant and socio-technical stance	24
4.5.2	Coding process	25
4.5.3	Theoretical sampling and saturation	25
4.5.4	Bloom's taxonomy as a sensitising concept	25
4.6	Rigour and trustworthiness	26
4.7	Ethics and data management	26
4.8	Methodological limitations	26
	Bibliography	27

LIST OF FIGURES

LIST OF TABLES

INTRODUCTION

1.1 Motivation

Software systems are complex socio-technical artefacts that must evolve under technical, organisational, and human constraints. In this setting, software and conceptual modelling provide structured abstractions that can support reasoning, communication, and early validation before implementation. In higher education, modelling is therefore commonly taught in software engineering and related programmes through notations and methods such as UML, BPMN, ER, or goal-oriented approaches.

Despite its recognised importance, modelling is frequently perceived by students in polarised ways: as a valuable instrument for understanding and communicating about systems, or as burdensome “extra documentation” that competes with implementation time. These perceptions are not only individual preferences; they can be shaped by socio-technical conditions such as the modelling tools used, the authenticity and domain of assignments, the form and timing of feedback, and the roles that models play in a course (e.g., communication versus strict code generation). Understanding how such factors interact with conceptual and procedural aspects of modelling is important for improving modelling education and for fostering modelling competence that transfers beyond individual courses.

1.2 Problem statement and research gap

Research on modelling education has produced valuable insights, but it is dispersed across modelling families and research communities. Landscape studies highlight fragmentation in terminology, theoretical grounding, and evaluation approaches across subdomains. In parallel, Bloom-based frameworks and modelling-specific taxonomies provide systematic ways to describe modelling learning outcomes and competencies, often indicating imbalances such as underrepresentation of evaluation and metacognitive aspects. Empirical studies of particular modelling methods further show that beginners face challenges

spanning construct semantics, application to requirements, and model quality judgement, and that these challenges can be linked to specific pedagogical and tool-supported requirements.

However, there remains a need to consolidate these perspectives into an integrated understanding of modelling education challenges and support needs that reflects both student and instructor viewpoints across educational contexts.

1.3 Aim and research questions

The aim of this thesis is to develop an empirically grounded account of modelling education challenges and support needs in higher education, treating modelling as a socio-technical activity that combines conceptual understanding, procedural strategies, and contextual influences.

The study is guided by the following research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across different modelling activities (e.g., interpreting, creating, refining, and evaluating models)?
- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

1.4 Research approach overview

To address these questions, the thesis adopts a qualitative, theory-building approach based on Grounded Theory, informed by Socio-Technical Grounded Theory to explicitly account for interactions between social and technical elements. Data are collected through semi-structured interviews with students and instructors complemented by activity-centered elicitation anchored in concrete modelling tasks or artefacts. This design is intended to elicit both reflective accounts and situated examples of modelling practice and difficulties. Bloom's revised taxonomy is used as a sensitising concept to support interpretation and communication of the cognitive nature of challenges, without constraining the grounded analysis.

1.5 Expected contributions

1.6 Thesis structure

The remainder of this thesis is structured as follows. Chapter 2 provides background on software and conceptual modelling, modelling education, grounded theory, Bloom-based learning outcomes, and human factors relevant to modelling learning. Chapter 3 reviews related work on modelling education frameworks, empirical studies of modelling learning challenges, tooling and model-driven engineering in education, and contextual factors such as motivation, inclusion, and authenticity. Chapter 4 presents the research design, research questions, data collection and analysis procedures, and considerations of rigour, ethics, and limitations. Subsequent chapters present the findings and discuss implications for modelling education and tool support.

BACKGROUND

2.1 Software modelling & conceptual modelling

2.1.1 Purpose and role of modelling in software engineering

Software systems are complex socio-technical constructs that must satisfy functional and non-functional requirements while evolving under technical, organisational, and human constraints. Within this context, software modelling supports reasoning about software systems by providing abstractions that capture relevant aspects of a domain or system at an appropriate level of detail. Models can serve multiple roles: they may act as design blueprints, facilitate communication among stakeholders, and enable analysis and validation of properties before implementation [13, 5].

Although modelling is often introduced through widely adopted notations such as UML or BPMN, it should not be reduced to “drawing diagrams”. Instead, models can be understood as structured representations expressed in a modelling language, grounded in that language’s semantics, and created for a specific purpose and audience. From this perspective, modelling is not merely documentation; it is a way of constructing and manipulating knowledge about a domain or system [5].

2.1.2 Conceptual modelling as a broader discipline

It is useful to distinguish between software modelling as a practice within software engineering and conceptual modelling as a broader discipline concerned with representing, structuring, and reasoning about domains and systems across fields. Conceptual modelling is not limited to software development alone; it is also used in organisational analysis, enterprise and service design, data-intensive systems, and other domains where abstraction is needed to manage complexity and support understanding [13]. Reviews of conceptual modelling education underline its interdisciplinary nature and identify a broad family of modelling subtypes—including data, process, goal, and enterprise modelling—that often evolve in parallel, with limited pedagogical or theoretical integration across subfields [13, 3, 2].

This disciplinary perspective is particularly relevant for research on teaching and learning software modelling. When modelling is treated primarily as a supplementary topic within software engineering courses, students may not fully appreciate its significance and may develop oversimplified views of its purpose and applicability. In contrast, the conceptual modelling perspective frames modelling languages as knowledge schemas: they define what can be expressed, how meaning is structured, and what kinds of analysis or tooling become possible. Understanding modelling therefore requires engaging with notational elements and underlying semantics, as well as modelling goals and the intended audience of the produced model artefacts [5].

2.1.3 Modelling methods, languages, and artefacts

Several authors emphasise that a modelling method is more than a set of symbols or a notation. A modelling method can be understood as comprising a modelling language (including syntax, semantics, and notation), a modelling procedure (the process or steps to create models), and supporting mechanisms and algorithms such as analysis, simulation, or transformation capabilities [4, 5]. This holistic view clarifies why students may be able to reproduce diagrammatic symbols yet still struggle to apply modelling meaningfully: the procedural and analytical aspects of modelling are often less explicitly taught or assessed.

From this viewpoint, the value of a modelling language is not only in enabling representation, but also in supporting reasoning. When a language is treated as a structured schema, models become more than static diagrams: they can become queryable, analysable, and transformable artefacts that enable deeper engagement with a domain. This also motivates tool support that can provide feedback on syntactic correctness, highlight semantic issues, help manage complexity, or automate parts of modelling tasks [2, 13].

2.1.4 Implications for the scope of this thesis

This thesis adopts the view that software modelling is an instantiation of the broader discipline of conceptual modelling. It therefore treats modelling as a discipline involving conceptual understanding of modelling constructs and their semantics, procedural competence in constructing and refining model artefacts from problem descriptions, analytical judgement of model quality and fit-for-purpose, and socio-technical awareness of how modelling practices are shaped by tools, learning contexts, and stakeholder needs. This framing aligns with work on modelling learning outcomes and competencies, which emphasises not only notational knowledge but also procedural skills, evaluative abilities, and reflective understanding [2, 3, 4, 14]. This perspective motivates the subsequent background on modelling education, grounded theory, and learning outcomes, and provides a foundation for investigating modelling challenges from both student and instructor perspectives.

2.2 Modelling education

2.2.1 Overview of conceptual modelling education

Conceptual and software modelling are widely taught across computing and software-related programmes, but the way these topics are taught and integrated into curricula remains fragmented. Maslov et al. [13] map the conceptual modelling education literature and show that educational work is scattered across various modelling sub-communities (e.g., data modelling, process modelling, enterprise modelling, etc.), which often evolve in parallel with limited reuse of frameworks, terminology, and learning theories across subfields [13]. Within this landscape, modelling appears in diverse course contexts (from introductory programming courses to advanced software engineering), and courses emphasise different modelling families and purposes (e.g., data-centric, process-centric, goal-oriented, etc.).

The same studies also highlight that much of the current practice in modelling education is driven by instructor experience and preferences, local institutional constraints, and the availability of tools, rather than by systematically articulated educational frameworks or empirically validated design principles [13]. While there is a broad consensus on the importance of modelling skills for software professionals, there is less agreement on which specific *competencies* learners should acquire, how these competencies should be scaffolded, and how learning should be assessed in a reliable and educationally meaningful way.

2.2.2 Bloom-based perspectives on modelling learning outcomes

In response to the fragmented state of modelling education, several authors have proposed to adapt Bloom’s revised taxonomy to conceptual and domain modelling, with the goal of making learning outcomes more explicit and comparable across courses and contexts. At a general level, these works start from the cognitive levels of remembering, understanding, applying, analysing, evaluating, and creating [9], and specialise them to the kinds of activities involved in modelling, such as interpreting models, constructing models from textual descriptions, critiquing model quality, or designing new modelling methods.

Bogdanova and Snoeck analyse learning outcomes and assessment tasks for domain modelling through the lens of Bloom’s revised taxonomy, identifying that existing tasks tend to concentrate on understanding, analysing, and creating conceptual knowledge while giving less attention to evaluation, procedural knowledge, or metacognition [3]. Building on this analysis, CaMeLOT provides a three-dimensional framework that combines Bloom’s cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and specific content areas of conceptual data modelling (e.g., classes, relationships, methods, quality, and tool support) [2]. This framework is instantiated with detailed example learning outcomes and can be used to design or audit courses.

Bork extends this Bloom-based perspective to both conceptual modelling and meta-modeling, mapping knowledge and cognitive dimensions to all components of a modelling

method (language, semantics, notation, procedure, mechanisms and algorithms) [4]. An evaluation of a “Smart City” teaching case using this framework reveals that even rich, practice-oriented material may still underemphasise certain cognitive levels (e.g., evaluation) or knowledge types (e.g., metacognitive knowledge). These works collectively suggest that Bloom’s taxonomy can provide a useful lens for reflecting on and improving modelling education, by making learning objectives more explicit and balanced across different dimensions.

2.2.3 Process and goal modelling education

Beyond data and structural modelling, there is growing interest in how students learn process and goal-oriented modelling. For business process modelling with BPMN, Maslov outlines a research agenda for developing empirically validated process-modelling education that integrates insights from conceptual modelling education, process model quality research, and learning sciences [18]. This agenda highlights the need for systematic mapping of existing work, empirical studies of novice modellers’ behaviour and typical errors, and the design and experimental evaluation of BPMN-specific teaching strategies and feedback mechanisms. This reflects a broader recognition that teaching process modelling effectively requires more than just introducing notation; it requires attention to cognitive demands, quality criteria, and the kinds of feedback needed to help learners improve.

Goal and actor-oriented modelling introduces additional layers of complexity, as it requires learners to reason about stakeholder intentions, rationales, and dependencies. The iStar learning study explicitly investigates the challenges students face when learning iStar, using a combination of Bloom-based task framing and grounded theory analysis of tutorials, modelling sessions, and interviews [19]. The authors report difficulties at multiple cognitive levels, including understanding the semantics of iStar constructs, applying the notation to real requirements, and analysing models to assess completeness and appropriateness. They derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, visualisation, and semi-automation modelling from text) that could help address these challenges [19].

These strands of work on BPMN and iStar education illustrate that different modelling families pose distinct learning challenges and may require specialised pedagogical approaches. They also reinforce the broader themes identified in conceptual modelling education: the need for scaffolding from text to model, support for managing complexity, and explicit attention to how learners evaluate and refine their models.

2.2.4 Modelling, tools, and model-driven engineering in education

Modelling in educational settings is often mediated by tools, ranging from simple drawing tools and web-based modellers to industrial-grade model-driven engineering (MDE) environments. Tool choice can influence students’ perceptions of modelling and their opportunities to practice particular skills. Liebel et al. conducted a multi-case study of

model-driven software engineering courses that use different tools (e.g., Umple, Papyrus) and place different emphasis on model-driven architecture and code generation [10]. Their results indicate that students' perceptions of tools are highly context-dependent: the same tool may be perceived positively in one course setup and negatively in another, depending on project design, available support, and the roles that models play (e.g., for communication versus strict code generation). They also find that perceptions of UML as modelling language are more positive and robust when models are used for requirements and analysis, rather than only as precise inputs to code generators [10].

These findings align with broader discussions of human factors in model-driven engineering, which argue that educational tools should prioritise usability, cognitive support, constructive feedback, and alignment with learning goals over industrial feature completeness [17]. From an educational perspective, this suggests that tool selection and integration should be carefully considered to ensure they support the intended modelling competencies and do not inadvertently increase cognitive load or distract from learning objectives.

2.2.5 Competence-oriented and human-centred perspectives

Broader software engineering education research brings in complementary competence-oriented and human-centred perspectives that can inform modelling education. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, arguing that curricula should be grounded in explicit competence models rather than only topic lists [14]. They introduce a six-level classification system (remember, understand, explain, use, apply, develop) for technical competencies and outline an iterative, data-driven approach to relating teaching methods and structural conditions to competency development [14]. This approach resonates with the Bloom-based frameworks discussed earlier, but emphasises the need for empirical validation and alignment with real-world professional requirements.

Work on human factors in modelling and software engineering education further highlights the role of motivation, inclusion, and authenticity. For example, studies on domain diversity and motivation in modelling courses show that educators' assumptions about which domains are motivating (e.g., games, automation) do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [16]. Students also report that feeling excluded by unfamiliar or culturally distant domains, or by assignment setups that do not fit their backgrounds, can hinder their engagement and learning [16]. More generally, experiential education initiatives emphasise the value of working on real systems, real tasks, and with real mentors to bridge the gap between classroom modelling exercises and the complexities of practice [7].

Taken together, these competence-oriented and human-centred perspectives reinforce the idea that modelling education should be designed not only around notations and

tools, but also around the competencies to be developed, the diversity of learners, and the authenticity and inclusiveness of modelling tasks and domains.

2.2.6 Implications for this thesis

The literature on modelling education, Bloom-based frameworks, process and goal modelling, tooling, competencies, and human factors provides several important implications for this thesis. First, it suggests that modelling learning outcomes can be systematically described in terms of cognitive and knowledge dimensions, and that existing courses may underemphasise evaluation and metacognitive aspects of modelling competence [3, 2, 4]. Second, it shows that different modelling families (e.g., data, process, goal) pose distinct learning challenges, and that empirical studies such as the iStar learning work can be used to derive grounded requirements for educational support [19, 18]. Third, it highlights that tools and model-driven approaches can both enable and hinder learning, depending on how they are integrated into course designs and on the kind of feedback and affordances they provide [10, 17]. Finally, competence-oriented and human-centred research indicates that modelling education must be sensitive to students' motivations, backgrounds, and learning environments, and that explicit competence models can support more systematic design and evaluation of teaching.

These insights inform the research questions and methodological choices of this thesis, which aims to investigate how students and teachers experience modelling, which challenges they perceive, and what kinds of support may foster the development of modelling competencies in realistic educational contexts.

2.3 Grounded Theory

2.3.1 Overview and key characteristics

Grounded Theory (GT) is a qualitative research methodology originally developed in sociology with the aim of generating theory that is systematically grounded in data rather than deduced from prior hypotheses. Instead of starting from a fixed theoretical framework, GT proceeds iteratively: data collection and analysis occur simultaneously, and emerging insights guide subsequent sampling and questioning. Core practices include *coding* (assigning conceptual labels to segments of data), *constant comparison* (comparing new data with existing codes and categories), and *theoretical sampling* (collecting new data to refine or challenge emerging concepts), and extensive *memoing* to capture analytic ideas [15, 1].

The outcome of a GT study is not a set of descriptive themes, but a more abstract, integrated explanation of how participants handle a central problem or concern. This explanation is typically organised around a *core category* that captures a dominant pattern in the data and links other categories together. Evaluation criteria emphasise the *fit* of concepts to the data, the *workability* of the emerging theory in explaining the studied

phenomenon, its *relevance* to practitioners and researchers, and its *modifiability* in light of new data, rather than statistical notions of validity [1].

Over time, GT has evolved into several variants/flavours, including *Classic GT* (Glaser and Strauss), *Straussian GT* (Strauss and Corbin), and *Constructivist GT* (Charmaz). While these variants share core principles, they differ in the degree to which they prescribe specific coding procedures, the role of prior theory, and the epistemological stance of the researcher. For software engineering research, this diversity offers flexibility but also introduces risks of inconsistent or superficial use if methodological choices are not made explicit [15].

2.3.2 Grounded Theory in software engineering

In software engineering (SE), GT has been used to study topics that are difficult to capture through controlled experiments or surveys, such as developer experience, team practices, decision-making processes, and socio-technical phenomena. A critical review by Stol et al. [15] finds that GT has become increasingly popular in SE research, but that many studies lack methodological rigour, adopt only parts of GT procedures, and show limited reflexivity regarding the researcher's role [15]. To address these issues, they offer a set of guidelines on when GT is appropriate (e.g., for under-theorised, practice-oriented problems), how to choose among GT variants, and how to report GT studies in a way that allows readers to assess their quality.

Adolph et al. provide a concrete example of applying classical (Glaserian) GT to study how software practitioners cope with the process of development, while also reflecting on the practical challenges of conducting GT research in SE contexts [1]. They describe in detail how they conducted open coding on interviews and field observations, developed concepts and categories through constant comparison, and iteratively refined their sampling strategy to explore interesting phenomena in more depth. Their work highlights the importance of staying close to the data, avoiding overly mechanical use of qualitative analysis tools, and systematically documenting analytic decisions through memos.

These contributions establish GT as a suitable methodology for SE when the research goal is to generate mid-level substantive theory about human and organisational aspects of software development, and they provide concrete guidance for designing and reporting such studies.

2.3.3 Socio-Technical Grounded Theory

Hoda et al. propose Socio-Technical Grounded Theory (STGT) as a specialisation of GT for SE that explicitly foregrounds the socio-technical nature of software development [6]. STGT retains the core principles of GT (iterative data collection and analysis, constant comparison, theoretical sampling, memoing) but emphasises the need to attend simultaneously to social and technical elements and their interactions. This involves, for example, paying attention

to roles, norms, tools, artefacts, and organisational structures, and analysing how these co-evolve over time.

STGT also articulates expectations for rigour in terms of credibility, transferability, dependability, and confirmability, and provides detailed guidance on reporting, including the description of research context, participants, data sources, coding procedures, and the evolution of categories and core concepts [6]. Therefore, STGT offers SE researchers a reporting template tailored to the realities of socio-technical systems.

2.3.4 Implications for this thesis

This thesis investigates how students and teachers experience software and conceptual modelling, and how they perceive challenges, learning processes, and support needs. These are socio-technical questions that involve individual cognition, educational practices, tools, and organisational constraints, and they are not yet well theorised in the software modelling education literature. Grounded Theory is therefore an appropriate methodological choice for this work.

2.4 Bloom's Taxonomy and learning outcomes

2.4.1 Overview of Bloom's revised taxonomy

The original Bloom's taxonomy was presented as a hierarchical classification of educational objectives, describing increasingly complex cognitive activities from simple recall to higher-order thinking. Krathwohl's revision restructures the taxonomy into a two-dimensional framework that distinguishes between a *knowledge dimension* (factual, conceptual, procedural, metacognitive) and a *cognitive process dimension* (remember, understand, apply, analyse, evaluate, create) [9]. This revised taxonomy encourages educators to formulate learning outcomes that are explicit both about the type of knowledge involved and the level of cognitive processing expected from learners.

In the revised framework, *factual knowledge* concerns basic elements that students must know, such as terminology and specific details; *conceptual knowledge* involves relationships among elements and larger structures; *procedural knowledge* relates to methods, techniques, and criteria for using procedures; and *metacognitive knowledge* refers to awareness of one's own cognition and learning strategies. The cognitive process dimension ranges from lower-order processes such as remembering and understanding, through applying and analysing, to higher-order processes of evaluating and creating [9]. The combination of these two dimensions supports a more fine-grained articulation of what learners should know and be able to do.

2.4.2 Learning outcomes in computing and software engineering

In computing and software engineering education, Bloom's revised taxonomy has been widely adopted as a tool for structuring curricula, designing assessments, and aligning teaching with intended learning outcomes. Typical applications include mapping course objectives to cognition levels, ensuring that programmes do not over-concentrate on lower-order outcomes, and checking that assessment tasks actually require the intended level of performance. However, generic formulations of the taxonomy can be difficult to apply directly in specialised domains such as software modelling, where domain-specific activities (e.g., interpreting a diagram, constructing a model from text, critiquing a modelling method) require more tailored descriptions.

This has led to several domain-specific adaptations of Bloom's taxonomy in areas such as domain modelling, conceptual data modelling, and metamodeling [3, 2, 4]. These adaptations retain the underlying cognitive-process levels while refining the knowledge categories and example tasks to reflect the particular demands of modelling work.

2.4.3 Bloom-based formulations of modelling learning outcomes

Several works in the modelling education literature explicitly build on Bloom's revised taxonomy to describe and analyse modelling-related learning outcomes. Bogdanova and Snoeck adapt the taxonomy to domain modelling by specifying what each cognitive level can mean in this context, such as remembering notation elements, understanding the meaning of relationships, applying modelling rules to construct diagrams, analysing models for completeness and consistency, evaluating alternative designs, and creating new models from complex descriptions [3]. Using this adapted taxonomy, they classify a large set of assessment tasks and show that existing practice tends to focus on understanding, analysing, and creating conceptual knowledge, with comparatively little emphasis on evaluation tasks, procedural knowledge, or metacognitive learning outcomes [3].

Extending this work, CaMeLOT proposes a three-dimensional framework for conceptual data modelling that explicitly combines Bloom's cognitive levels with the knowledge dimension and specific content areas such as classes, relationships, constraints, modelling methods, quality criteria, and tool usage [2]. The authors instantiate this framework with example learning outcomes, demonstrating how it can be used to design, review, or compare modelling courses. Their analysis also suggests that metacognitive outcomes (e.g., reflecting on modelling strategies, recognising one's own limitations) are rarely made explicit in current curricula [2].

Bork further generalises the Bloom-based perspective by developing separate yet related taxonomies for conceptual modelling and metamodeling [4]. He maps the knowledge and cognitive dimensions to the components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) and uses this mapping to evaluate a Smart City teaching case. The evaluation reveals an imbalance: many activities target creation and application at higher cognitive levels, while explicit evaluate-level tasks and

metacognitive activities are largely absent [4]. This pattern is consistent with the findings of Bogdanova and Snoeck and suggests that modelling education often emphasises the ability to produce models but gives less structured attention to helping students judge model quality or reflect on their modelling practice.

These Bloom-based formulations show that learning outcomes for modelling can be described systematically across different levels and types of knowledge, and they provide concrete examples of how to manage this in educational settings. They also highlight recurring gaps, particularly in relation to evaluative and metacognitive aspects of modelling competence.

2.4.4 Bloom, competencies, and this thesis

For this thesis, Bloom’s revised taxonomy and its modelling-specific adaptations serve two main purposes. First, they offer a conceptual lens for thinking about modelling-related learning outcomes and competencies, distinguishing, for example, between understanding modelling constructs, applying modelling procedures, analysing and evaluating models, and reflecting on one’s own modelling practice. Second, the documented imbalances in current modelling education (e.g., underrepresentation of evaluation and metacognitive outcomes) motivate a closer examination of how students and teachers experience modelling and what kinds of support they perceive as needed [3, 2, 4, 14].

2.5 Human factors in modelling and SE education

2.5.1 Motivation, engagement, and the “value” of modelling

Software and conceptual modelling are often perceived by students as either highly valuable (e.g., for understanding complex systems and communicating design intent) or as burdensome “extra documentation” that competes with implementation time [16]. Such perceptions are not merely a matter of individual preference; they reflect a broader set of human factors that shape engagement, learning effort, and ultimately competence development. In modelling education, learners frequently report that motivation depends on whether modelling tasks feel authentic, whether their outputs matter beyond grading, and whether modelling is integrated into a meaningful workflow rather than treated as a purely academic exercise.

Research on modelling education explicitly highlights motivation as a key factor. A study on domain diversity, motivation, and inclusion in software modelling education shows that educators’ assumptions about which domains are motivating do not always match students’ preferences [16]. Students tend to value domains that connect to their interests, everyday experiences, or socially relevant contexts, and they report higher motivation when they have some agency (e.g., choice or personalisation of the domain) [16]. These results suggest that human factors in modelling education cannot be separated

from how modelling tasks are framed, which domains they use, and whether students can see a clear purpose for modelling beyond producing a diagram.

2.5.2 Inclusion, domain diversity, and participation barriers

Human factors also include issues of inclusion and participation. Even when modelling topics are formally accessible, examples and assignments can implicitly privilege specific backgrounds, interests, or prior experiences. Domain-diversity research indicates that certain commonly used modelling domains (e.g., video games or specific technical automation scenarios) may be engaging for some students but alienating for others [16]. Students may also experience barriers related to group-work structures, competitive gamification elements, or assumptions about familiarity with particular cultural or socioeconomic contexts. These factors can influence not only motivation but also opportunities for practice and feedback, thereby affecting learning outcomes.

From a course-design perspective, these findings motivate the careful selection of modelling domains and assignment setups, balancing instructors' preferences and curricular relevance with the diversity of student experiences and interests. Providing alternative domains, allowing domain choice, and ensuring that examples do not rely on narrow cultural references can be considered human-centred strategies to improve participation and reduce disengagement [16].

2.5.3 Tools as mediators of learning experience

In modelling education, tools play a critical role as mediators of the learning experience. They actively shape how students interact with modelling concepts, including how easily they can experiment, receive feedback, manage complexity, and recover from errors. Usability issues, steep learning curves, or misalignment between tool features and learning objectives can lead to frustration and negative attitudes toward modelling, even when the underlying notation is conceptually useful.

Liebel et al. examine model-driven software engineering education through a multi-case study and report that students' perceptions of modelling tools are highly context-dependent [10]. The same tool may be perceived as useful or cumbersome depending on how it is integrated into the course design, what the modelling artefacts are used for (e.g., communication vs. code generation), and the level of support provided. They also find that students tend to have more positive perceptions of UML when models are used for analysis and communication, even if tool-related issues reduce satisfaction in code-generation contexts [10]. This suggests that negative experiences are frequently tied to the socio-technical configuration of the course (tool, task, expectations) rather than to modelling as a concept.

2.5.4 Authenticity, experiential learning, and social dynamics

Human factors in SE education also encompass the extent to which learning activities resemble real-world practice. Experiential education initiatives emphasise that authenticity can enhance motivation, engagement, strengthen soft-skill development, and improve students' understanding of the complexities of software practice. Holmes et al. identify three "dimensions of experientialism" in a large, distributed capstone programme: working on real projects (existing open-source systems and codebases), performing real tasks (non-trivial contributions), and engaging with real mentors (experienced developers embedded in the project) [7]. Students report that this combination helps them foster communication skills, teamwork, time management, and practical problem-solving abilities, while also deepening their technical understanding in authentic contexts [7]. Although modelling courses may not always be structured as capstones, the experientialism perspective is still relevant.

2.5.5 Competence development and human-centred educational design

Competence-oriented research in SE education argues that curricula should be grounded in explicit competence models of what students should be able to do, and that teaching methods and structural conditions should be linked to competence outcomes rather than treated as ad hoc choices [14]. Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing required competencies in software engineering, introducing a multi-level classification system for technical competencies intended to describe progression from basic familiarity to the ability to develop new solutions [14]. This approach emphasises iterative improvement through measuring competence levels, identifying influencing factors (structural conditions and process variables), and refining course designs accordingly [14].

Adopting such a competence-oriented and human-centred perspective in modelling education may require more than selecting an engaging domain or user-friendly tool; it may also involve designing learning activities and assessments that support the systematic progression across competencies, including conceptual understanding, procedural ability to create and refine models, evaluative judgement of model quality, and reflective awareness of modelling choices and limitations. The reviewed literature indicates evaluation and metacognitive competencies are often underemphasised in current modelling education [3, 2, 4], and human factors research provides a complementary explanation: learners may disengage when feedback is insufficient, tasks feel artificial, tools are frustrating, or domains do not resonate with their interests and experiences.

2.5.6 Implications for this thesis

The human factors literature motivates this thesis in several ways. First, it highlights that modelling education should be analysed not only in terms of notations and learning

outcomes, but also in terms of student and teacher experiences shaped by motivation, inclusion, feedback, and perceived authenticity [16, 7]. Second, modelling tools should be treated as socio-technical mediators that influence cognitive load, engagement, and attitudes toward modelling, making tool choice and integration important pedagogical determinants of learning experience [10, 17]. Third, competence-oriented perspectives suggest that empirical investigation should seek to connect perceived challenges and support needs to explicit competence development and course design choices [14].

RELATED WORK

3.1 Conceptual modelling education landscape

Existing work on conceptual modelling education spans multiple modelling families and research communities, including data modelling, process modelling, requirements-related modelling, and model-driven development. (However, the literature is not unified under a single pedagogical tradition.) A mapping of conceptual modelling education by Maslov et al. highlights that educational research is distributed across multiple communities and modelling subfields, which often evolve in parallel with limited reuse of terminology, learning theories, and evaluation approaches across subdomains [13]. This fragmentation complicates comparison of findings across modelling families and (limits the accumulation of shared evidence on which teaching strategies and supports are effective).

The present thesis builds on this observation by drawing together frameworks and empirical results from different modelling education strands (conceptual/data, process, goal-oriented, and tool-supported modelling) and by focusing on the specific challenges and support needs reported by both students and teachers.

3.2 Modelling frameworks

A common approach to strengthening modelling education research is to define explicit frameworks for learning outcomes, competencies, and assessment design. Several works adapt Bloom's revised taxonomy [9] to modelling activities, aiming to clarify what learners should know and be able to do beyond learning a notation. (The following subsections summarise three modelling-education frameworks that are especially relevant to this thesis.)

3.2.1 CaMeLOT: a framework for conceptual data modelling learning outcomes

CaMeLOT proposes an educational framework for conceptual data modelling that combines Bloom’s cognitive levels with a knowledge dimension (factual, conceptual, procedural, metacognitive) and modelling content areas [2]. The framework is instantiated with detailed example learning outcomes and can be used to design course objectives, learning activities, and assessments in a more systematic manner. (For this thesis, CaMeLOT is particularly valuable as a template for articulating modelling competence across cognitive and knowledge dimensions, and for motivating the inclusion of procedural and reflective elements rather than focusing exclusively on diagram construction.)

3.2.2 Domain Modelling in Bloom: empirical analysis of assessment practices

Bogdanova and Snoeck empirically analyse domain-modelling assessment tasks through the lens of Bloom’s revised taxonomy [3]. They report that many tasks concentrate on understanding, analysing, and creating conceptual knowledge, while evaluation, procedural knowledge, and metacognitive learning outcomes receive limited explicit attention. This work provides evidence that modelling courses may implicitly expect higher-level modelling competence without systematically scaffolding or assessing foundational skills. In this thesis, these findings support the motivation for examining modelling challenges in relation to cognitive demands and for considering whether educational support should target not only modelling syntax but also strategies for evaluation and refinement.

3.2.3 Teaching conceptual modelling and metamodeling

Bork extends Bloom-based framing to both conceptual modelling and metamodeling, relating cognitive and knowledge dimensions to the components of modelling methods (language, semantics, notation, procedure, mechanisms and algorithms) [4]. By analysing a teaching case using the proposed framework, the work suggests that even practice-oriented materials may strongly emphasise model creation while neglecting explicit evaluation tasks and metacognitive reflection. This perspective is useful for the present thesis because it frames modelling competence as method-level understanding and reflective judgement, aligning with this thesis focus on challenges experienced by learners and educators when moving from notation use to meaningful modelling practice.

3.3 Empirical studies on modelling learning challenges

While modelling frameworks clarify educational targets, empirical studies are needed to understand where learners struggle in practice and which supports are perceived as valuable. Two complementary strands are particularly relevant: studies of goal-oriented modelling learning challenges and research agendas for process modelling education.

3.3.1 iStar learning challenges and support requirements

The core empirical reference for this thesis is the iStar learning study, which investigates the challenges beginners face when learning and practicing iStar and derives requirements for facilitating learning [19]. Using Bloom-based task framing and grounded theory analysis, the study reports difficulties at multiple cognitive levels, including understanding the semantics of modelling constructs, applying the notation to real requirements scenarios, and analysing models for quality and completeness. The authors derive a catalogue of strategies and tool-supported requirements (e.g., conceptual clarification, syntax checking, process guidance, templates, advanced visualisation, and partial automation from textual requirements) and implement a subset of these requirements in a prototype tool [19]. This thesis builds on these insights by extending the focus from a single modelling method to broader modelling education contexts and by eliciting perspectives from both students and teachers.

3.3.2 Process modelling education and the need for empirical validation

In the domain of process modelling, Maslov argues for empirically validated process modelling education using BPMN and outlines a research agenda that integrates conceptual modelling education, process model quality research, and learning theory [18]. The agenda emphasises systematic synthesis of existing work, empirical investigation of novice modelling behaviour and typical errors, and the design and experimental validation of BPMN-specific teaching strategies and feedback mechanisms. (This work is relevant as a parallel to goal modelling education): it illustrates that modelling families pose distinct cognitive and practical demands, but that recurring educational needs include scaffolding, feedback, and mechanisms to manage complexity.

3.4 Tools and model-driven engineering in education

Modelling education is strongly shaped by tool choice, especially in model-driven and model-based engineering settings. Tools can support learners through feedback, automation, and manageability, but can also introduce friction that distorts students' perceptions of modelling.

3.4.1 Perception of tools and UML in MDE education

Liebel et al. conduct a multi-case study of model-driven software engineering courses and report that students' perceptions of modelling tools are highly context-dependent [10]. The same tool may be perceived as useful in one course and cumbersome in another depending on project structure, available support, and the role that models play (e.g., communication versus strict code generation). The study also suggests that perceptions of UML are more robustly positive when models are used for requirements and analysis

rather than exclusively as code generation inputs [10]. These results motivate the present thesis to treat tooling and course design as socio-technical factors that shape modelling experiences, rather than as neutral delivery mechanisms.

3.4.2 Human factors in model-driven engineering

Work on human factors in model-driven engineering argues for research that accounts for usability, cognitive load, and the alignment between tools and user goals [17]. From an educational perspective, this supports the view that modelling tools should be evaluated not only by feature completeness, but also by how they support learning processes such as exploration, error detection, reflection, and progressive mastery.

3.5 Human factors and learning context in modelling education

Beyond frameworks, methods, and tools, modelling education is influenced by motivational and contextual factors such as domain choice, inclusiveness, and the authenticity of tasks.

3.5.1 Domain diversity, motivation, and inclusion

Research on domain diversity and motivation in modelling education reports that educators' assumptions about "motivating" domains do not always match students' preferences, and that students value domains that connect to their interests and everyday or socially relevant contexts [16]. The work also highlights that assignment design can unintentionally exclude learners when domains or examples presume cultural, economic, or experiential background that not all students share [16]. These insights inform this thesis by motivating interview questions about domain choice, perceived relevance, and learning barriers that go beyond notation difficulties.

3.5.2 Experiential learning and authenticity

Holmes et al. propose dimensions of experientialism in software engineering education and show that students strongly value learning experiences that involve real projects, real tasks, and real mentors [7]. Although this work is not modelling-specific, it contributes a useful lens for interpreting how authenticity and mentorship can influence learners' engagement and competence development, and it supports the relevance of studying modelling experiences in realistic educational contexts.

3.6 Competence-oriented agendas and methodological precedents

Competence-oriented perspectives provide a complementary lens for structuring educational goals and for linking teaching approaches to measurable outcomes.

3.6.1 Competency identification and development in SE education

Sedelmaier and Landes propose the EVELIN research agenda for identifying and developing competencies in software engineering and argue that curricula should be grounded in explicit competence models rather than topic lists [14]. They introduce a six-level classification system for technical competencies and advocate an iterative approach in which competence targets are identified, competence levels are measured, and teaching strategies and structural constraints are adjusted over time [14]. While not specific to modelling, this agenda supports the thesis stance that modelling competence should be treated as multi-dimensional (conceptual, procedural, analytical, socio-technical) and studied through systematic empirical methods. It also complements Bloom-based modelling frameworks by emphasising explicit competence models and iterative alignment between competence targets, teaching methods, and structural constraints.

3.6.2 Grounded Theory in software engineering research

Methodologically, grounded theory has been widely discussed in software engineering research as an approach for developing explanatory theory from qualitative data. Stol et al. provide a critical review and practical guidelines for conducting grounded theory in software engineering [15], while Hoda et al. propose a socio-technical grounded theory perspective that explicitly accounts for the interplay between social and technical factors [6]. Adolph et al. offer a practical account of using grounded theory to study the experience of software development [1]. These works provide methodological grounding for the approach used in this thesis and support the validity of using interviews and iterative coding to derive themes and requirements from participant experiences.

3.7 Summary

In summary, existing work provides: (i) landscape-level evidence that modelling education research is fragmented across modelling families and communities [13]; (ii) frameworks that structure modelling learning outcomes using Bloom-based dimensions [3, 2, 4]; (iii) empirical studies and agendas that identify modelling-specific learning challenges and the need for validated educational support [19, 18]; and (iv) tool-, competence-, and human-centred perspectives that emphasise context, usability, motivation, inclusiveness, and authenticity [10, 16, 7, 14].

METHODOLOGY

4.1 Research design and rationale

This thesis investigates how software modelling is experienced, taught, and learned in higher education settings. The focus is on modelling as a socio-technical activity, encompassing conceptual understanding, procedural strategies, tool mediation, and educational and organisational constraints. Since these phenomena are not well explained by existing theory and are strongly context-dependent, the study adopts a qualitative, theory-building research design.

The study employs a Grounded Theory (GT) approach, which is well-suited to under-theorised problem areas where the goal is to generate an explanatory account grounded in empirical data such as participants' experiences and perspectives, rather than testing pre-existing hypotheses [15, 1]. To explicitly account for the interactions between social and technical elements (e.g., notations, tools, course structures, and stakeholder perspectives), the analysis is informed by Socio-Technical Grounded Theory (STGT) [6]. The study design follows an iterative logic in which data collection and analysis proceed in parallel, allowing emerging insights to guide subsequent sampling and questioning [1, 15].

In addition, the methodology draws inspiration from prior empirical studies on modelling education, particularly the iStar learning study, which combined task-based learning activities with semi-structured interviews and grounded theory analysis to derive a structured set of challenges and corresponding support requirements [19]. The present work adapts and extends this approach to a broader scope beyond a single modelling method, while maintaining the strengths of an activity-centered elicitation design and a systematic theory-building analysis.

4.2 Research questions

The study is guided by the following overarching research questions:

- **RQ1:** What challenges do students and instructors perceive in learning and teaching software modelling in higher education, and how do these challenges manifest across

different modelling activities (e.g., interpreting, creating, refining, and evaluating models)?

- **RQ2:** What strategies do students and instructors employ to address these challenges, and what support needs (pedagogical and tool-related) do they identify?
- **RQ3:** How do socio-technical factors (e.g., tools, course design, domains, feedback practices) shape modelling learning and teaching experiences?

These questions are exploratory and explanatory in nature, aiming to uncover the lived experiences of key stakeholders (students and instructors) and making them suitable for a grounded-theory-based methodology [15, 6].

4.3 Study context and participants

The study is conducted in the context of higher education software engineering and related courses that include software modelling components, such as requirements engineering, software design, business process modelling, or model-driven engineering. Two participant groups are targeted:

- **Students:** learners who have encountered modelling activities in at least one course and have completed at least one modelling-related assignment or project.
- **Instructors:** lecturers or teaching assistants who have taught modelling-related content and have experience assessing student models or guiding modelling activities.

Sampling follows the principle of theoretical sampling: initial participants are recruited to seed the analysis, and subsequent recruitment is guided by emerging categories (e.g., differences between courses, notations, or tool ecosystems) until categories are sufficiently elaborated [1, 15]. To ensure informative accounts, inclusion criteria emphasise participants with direct experience of modelling tasks rather than only theoretical exposure.

4.4 Data collection

Data collection consists of semi-structured interviews complemented by an activity-centered elicitation component. This combination is intended to elicit both reflective accounts and concrete, situated examples of modelling practice, following the logic used in prior modelling education research [19]. The procedure was refined through a pilot to ensure clarity, timing, and depth of questioning.

4.4.1 Instruments and pilot

Two semi-structured interview guides were developed: one for students and one for instructors. The guides share a common backbone (perceptions of modelling, challenges,

strategies, support needs) but differ in emphasis: student interviews focus on learning experiences and task-level difficulties, whereas instructor interviews focus on teaching goals, scaffolding, assessment, and observed misconceptions. The pilot interview phase was used to validate the interview flow, estimate duration (20–30 minutes), and improve probing questions for eliciting concrete examples and contextual detail.

Bloom-based modelling frameworks also informed the formulation of prompts by mapping modelling competence to observable activities (e.g., interpreting models, constructing models from text, and judging model quality), supporting a balanced coverage of cognitive demands without imposing Bloom as a coding template.

4.4.2 Activity-centered elicitation: modelling task

To move beyond abstract opinions, participants are invited to discuss a concrete modelling task or artefact. Depending on participant context, this includes one of the following: (i) a short modelling exercise based on a brief scenario, or (ii) a retrospective walkthrough of a recent modelling assignment or model created/assessed in a course.

This activity-centered elicitation is inspired by the iStar learning study, which used a tutorial and modelling session followed by an interview to anchor participant reflections in a concrete modelling experience [19]. The goal is to surface tacit reasoning, decision points, typical errors, and tool- or feedback-related constraints that may not emerge from interview questions alone.

4.4.3 Interviews

Interviews are conducted individually and last approximately 20–30 minutes. Student interviews cover background, perceived usefulness of modelling, learning difficulties, strategies for going from text to model, evaluation criteria, tool experiences, and motivational and contextual factors (including inclusion-related experiences where relevant). Instructor interviews address teaching objectives, scaffolding practices, assessment design, common student misconceptions, tool choices and constraints, and perceived gaps in current pedagogy. Interviews include flexible probing to explore emerging themes in depth, consistent with GT’s iterative and adaptive logic [1, 15].

4.4.4 Surveys

4.5 Data analysis

4.5.1 Grounded Theory variant and socio-technical stance

The analysis follows Grounded Theory principles as adapted for software engineering research [15, 1]. In particular, the study adopts a socio-technical lens aligned with STGT, treating modelling education as an interaction between social elements (students, instructors, feedback practices, course constraints) and technical elements (notations, tools,

model artefacts, automation features) [6]. This stance informs both the coding process and the interpretation of emerging categories.

4.5.2 Coding process

The coding process follows an iterative progression of open coding, axial coding, and selective coding. Open coding is used to label segments of interview transcripts and artefacts with conceptual codes capturing modelling challenges, strategies, perceptions of value, tool interactions, and contextual constraints. Constant comparison is applied throughout: newly coded incidents are compared to previous incidents and codes to refine categories and identify variation [1].

Axial coding is used to relate categories using a coding paradigm (e.g., phenomenon, causal conditions, context, intervening conditions, and strategies), as demonstrated in modelling education research on iStar learning [19]. Selective coding is then employed to integrate categories into a coherent explanatory account. Following guidance for GT in SE, memoing is used continuously to document analytic decisions, emerging hypotheses, and links between categories [15, 1].

4.5.3 Theoretical sampling and saturation

Sampling decisions are guided by emerging categories rather than predetermined demographic quotas. For example, if early analysis suggests that tool frustration is a major driver of negative modelling attitudes, additional participants may be recruited from courses with different tool ecosystems to explore variation. Data collection continues until categories are sufficiently elaborated and additional data no longer contribute new properties or relationships to the core explanatory account [1, 15].

4.5.4 Bloom's taxonomy as a sensitising concept

Bloom's revised taxonomy is used as a sensitising concept to support interpretation and communication of findings, particularly when describing the cognitive nature of modelling challenges (e.g., understanding modelling constructs, applying modelling procedures, analysing model quality) [9]. Importantly, Bloom is not imposed as a rigid coding template; instead, grounded categories remain primary, and Bloom-inspired language is used where it aligns with participant accounts and helps structure the presentation of results, consistent with GT guidance to avoid forcing data into preconceived frameworks [15].

4.6 Rigour and trustworthiness

4.7 Ethics and data management

Participation is voluntary and based on informed consent. Interview recordings and transcripts are anonymised, and any potentially identifying details (e.g., course identifiers, project names, personal data) are removed or generalised. Data are stored securely and used only for the purposes of this research. When collecting modelling artefacts (e.g., student models or assignments), additional care is taken to remove identifying information and to ensure that artefacts are used in ways consistent with participants' consent and institutional policies.

4.8 Methodological limitations

As a qualitative, context-sensitive study, the findings aim for analytical rather than statistical generalisation. Participant accounts may be influenced by recall bias, social desirability, and variation in course contexts. Additionally, tool- and notation-specific experiences may limit transferability across institutions or modelling families. These limitations are mitigated by triangulating accounts with concrete artefacts where possible, sampling for variation as categories emerge, and explicitly documenting contextual factors that shape the observed phenomena [15, 6].

BIBLIOGRAPHY

- [1] S. Adolph, W. Hall, and P. Kruchten. “Using Grounded Theory to Study the Experience of Software Development”. In: *IEEE Software* 28.4 (2011), pp. 24–27 (cit. on pp. 9, 10, 21–25).
- [2] D. Bogdanova and M. Snoeck. “CaMeLOT: An Educational Framework for Conceptual Data Modelling”. In: *TBD* (2020) (cit. on pp. 4–6, 9, 12, 13, 15, 18, 21).
- [3] D. Bogdanova and M. Snoeck. “Domain Modelling in Bloom: Deciphering How We Teach It”. In: *TBD* (2020) (cit. on pp. 4–6, 9, 12, 13, 15, 18, 21).
- [4] D. Bork. “A Framework for Teaching Conceptual Modeling and Metamodeling Based on Bloom’s Revised Taxonomy”. In: *TBD*. 2020 (cit. on pp. 5, 7, 9, 12, 13, 15, 18, 21).
- [5] R. A. Buchmann, D. Karagiannis, and H.-G. Fill. “Conceptual Modelling in Education: A Position Paper”. In: *TBD*. 2020 (cit. on pp. 4, 5).
- [6] R. Hoda, J. Noble, and S. Marshall. “Socio-Technical Grounded Theory for Software Engineering”. In: *IEEE Transactions on Software Engineering* 44.12 (2018), pp. 1208–1226 (cit. on pp. 10, 11, 21–23, 25, 26).
- [7] R. Holmes, R. J. Walker, and G. C. Murphy. “Dimensions of Experientialism for Software Engineering Education”. In: *TBD* (2014) (cit. on pp. 8, 15, 16, 20, 21).
- [8] B. Kitchenham and S. L. Pfleeger. “Teaching Survey Research in Software Engineering”. In: *ACM SIGSOFT Software Engineering Notes* 33.6 (2008), pp. 1–10.
- [9] D. R. Krathwohl. “A Revision of Bloom’s Taxonomy: An Overview”. In: *Theory Into Practice* 41.4 (2002), pp. 212–218 (cit. on pp. 6, 11, 17, 25).
- [10] G. Liebel et al. “Model Driven Software Engineering in Education: A Multi-Case Study on Perception of Tools and UML”. In: *TBD* (2014) (cit. on pp. 8, 9, 14, 16, 19–21).
- [11] J. Linaker, P. Runeson, and B. Regnell. “Challenges in Survey Research”. In: *Empirical Software Engineering* 20.4 (2015), pp. 924–962.

- [12] J. M. Lourenço. *The NOVAtesis L^AT_EX Template User's Manual*. NOVA University Lisbon. 2021. URL: <https://github.com/joaomlourenco/novathesis/raw/main/template.pdf>.
- [13] A. Maslov, S. Poelmans, and N. Rosenthal. "Conceptual Modeling Education". In: *TBD* (2021) (cit. on pp. 4–6, 17, 21).
- [14] Y. Sedelmaier and D. Landes. "A Research Agenda for Identifying and Developing Required Competencies in Software Engineering". In: *TBD* (2014) (cit. on pp. 5, 8, 13, 15, 16, 21).
- [15] K.-J. Stol, P. Ralph, and B. Fitzgerald. "Grounded Theory in Software Engineering Research: A Critical Review and Guidelines". In: *IEEE Transactions on Software Engineering* 42.12 (2016), pp. 1201–1222 (cit. on pp. 9, 10, 21–26).
- [16] TBD. "Domain Diversity, Motivation, and Inclusion in Software Modelling Education". In: *TBD*. 2020 (cit. on pp. 8, 13, 14, 16, 20, 21).
- [17] TBD. "Human Factors in Model-Driven Engineering: Future Research Goals and Initiatives for MDE". In: *TBD*. 2020 (cit. on pp. 8, 9, 16, 20).
- [18] TBD. "Towards Empirically Validated Process Modelling Education Using a BPMN Formalism". In: *TBD*. 2020 (cit. on pp. 7, 9, 19, 21).
- [19] TBD. "Understanding the Challenges and Requirements for Facilitating iStar Learning: An Empirical Study with iStar Learners". In: *TBD*. 2017 (cit. on pp. 7, 9, 19, 21–25).

